Ex 5	Visualisation of textual data
Date	

Aim:

To use textual data to perform text preprocessing techniques and analyse the processed data through appropriate data visualisation methods in order to gain meaningful insights.

Software Used

• Programming Language: Python 3.x

• Environment: Jupyter Notebook.

Description:

The system will perform the following tasks:

1. **Text Preprocessing:** Tokenize the text, remove stop words, and perform lemmatization.
2. **Word Frequency Analysis:** Identify the most common words in the text documents.
3. **Sentiment Analysis:** Determine the sentiment (positive, neutral, negative) of the documents.
4. **Topic Modeling:** Use Latent Dirichlet Allocation (LDA) to uncover hidden topics in the documents.
5. **Visualization:** Generate various visualizations, such as word clouds, bar charts for word frequencies, and pie charts for sentiment analysis.

Dataset details

The dataset includes the following columns:

- **reviewerID -** Unique ID of the reviewer (customer).
- **asin -** Amazon Standard Identification Number – unique product ID.
- **reviewerName -** Name of the reviewer (may be missing/NaN).
- **helpful -** A list showing helpful votes in the format: [helpful_yes, total_votes]
- **reviewText -** The full text written by the reviewer.
- **overall -** Star rating given by the reviewer (1 to 5).
- **summary -** Short title or summary of the review.
- **unixReviewTime -** Review time in Unix timestamp format.
- **reviewTime -** Review date in readable format (YYYY-MM-DD).
- **day_diff -** Number of days between the review date and a reference date (used for recency analysis).
- **helpful_yes -** Number of users who marked the review as helpful.
- **total_vote -** Total number of votes received for helpfulness.

List of tasks

Interpret the results and identify the important insights for each task.

1. Load the review dataset into a pandas DataFrame and display the first few rows.

```
import pandas as pd
import matplotlib.pyplot as plt
import nltk
df = pd.read_csv("amazon_review.csv")
df.head()
```

OUTPUT:

	reviewerID	asin	reviewerName	helpful	reviewText	overall	summary	unixReviewTime	reviewTime	day_diff	helpful_yes	total_vote
0	A3SBTW3WS4IQSN	B007WTAJTO	NaN	[0, 0]	No issues.	4	Four Stars	1406073600	23-07-2014	138	0	0
1	A18K1ODH1I2MVB	B007WTAJTO	0mie	[0, 0]	Purchased this for my device, it worked as adv...	5	MOAR SPACE!!!	1382659200	25-10-2013	409	0	0
2	A2FII3I2MBMUIA	B007WTAJTO	1K3	[0, 0]	it works as expected. I should have sprung for...	4	nothing to really say....	1356220800	23-12-2012	715	0	0
3	A3H99DFEG68SR	B007WTAJTO	1m2	[0, 0]	This think has worked out great.Had a diff. br...	5	Great buy at this price!!! *** UPDATE	1384992000	21-11-2013	382	0	0
4	A375ZM4U047O79	B007WTAJTO	2&1/2Men	[0, 0]	Bought it with Retail Packaging, arrived legit...	5	best deal around	1373673600	13-07-2013	513	0	0

2. Tokenize the reviewText or summary column into individual words using NLTK's word_tokenize function.

```
nltk.download('punkt')
nltk.download('punkt_tab')

from nltk.tokenize import word_tokenize
df['reviewText']=df['reviewText'].fillna("")
df['tokens']=df['reviewText'].apply(word_tokenize)
print(df[['reviewText','tokens']].head())
```

OUTPUT:

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sumit\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sumit\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
True
```

3. Convert all text in reviewText and summary columns to lowercase for uniformity.

```
df['summary'] = df['summary'].astype(str).str.lower()
df['reviewText'] = df['reviewText'].astype(str).str.lower()
```

OUTPUT:

```
                                          reviewText  \
0                                          No issues.
1  Purchased this for my device, it worked as adv...
2  it works as expected. I should have sprung for...
3  This think has worked out great.Had a diff. br...
4  Bought it with Retail Packaging, arrived legit...

                                              tokens
0                                     [No, issues, .]
1  [Purchased, this, for, my, device, ,, it, work...
2  [it, works, as, expected, ., I, should, have, ...
3  [This, think, has, worked, out, great.Had, a, ...
4  [Bought, it, with, Retail, Packaging, ,, arriv...
```

4. Remove English stopwords from the reviewText and summary columns using NLTK's stopwords list.

```
import nltk
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
def remove_stopwords(text):
    tokens = word_tokenize(str(text))
    filtered = [word for word in tokens if word.lower() not in stop_words]
    return " ".join(filtered)
df['reviewText_no_stop'] = df['reviewText'].apply(remove_stopwords)
print(df[['reviewText', 'reviewText_no_stop']].head())
```

OUTPUT:

```
                                          reviewText  \
0                                          no issues.
1  purchased this for my device, it worked as adv...
2  it works as expected. i should have sprung for...
3  this think has worked out great.had a diff. br...
4  bought it with retail packaging, arrived legit...

                                 reviewText_no_stop
0                                          issues .
1  purchased device , worked advertised . never m...
2  works expected . sprung higher capacity . thin...
3  think worked great.had diff . bran 64gb card w...
4  bought retail packaging , arrived legit , oran...
```

5. Remove all punctuation from the reviewText and summary columns

```
import string
from nltk.tokenize import word_tokenize
punctuations = set(string.punctuation)
def rdfve_punctuation(text):
    tokens = word_tokenize(str(text))
    filtered_tokens = [word for word in tokens if word not in punctuations]
    return " ".join(filtered_tokens)
df['reviewText_clean'] = df['reviewText'].apply(rdfve_punctuation)
df['summary_clean'] = df['summary'].apply(rdfve_punctuation)
print(df[['reviewText', 'reviewText_clean', 'summary', 'summary_clean']].head())
```

OUTPUT:

```
                                        reviewText  \
0                                       no issues.
1  purchased this for my device, it worked as adv...
2  it works as expected. i should have sprung for...
3  this think has worked out great.had a diff. br...
4  bought it with retail packaging, arrived legit...

                                   reviewText_clean  \
0                                          no issues
1  purchased this for my device it worked as adve...
2  it works as expected i should have sprung for ...
3  this think has worked out great.had a diff bra...
4  bought it with retail packaging arrived legit ...

                                summary                    summary_clean
0                            four stars                       four stars
1                        moar space!!!                        moar space
2               nothing to really say....      nothing to really say ....
3  great buy at this price!!!  *** update  great buy at this price update
4                       best deal around                 best deal around
```

6. Create a WordCloud visualization for the summary column to display the most frequent words.

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text = " ".join(df['summary'])

wordcloud = WordCloud(width=800, height=400, background_color='black').generate(text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```

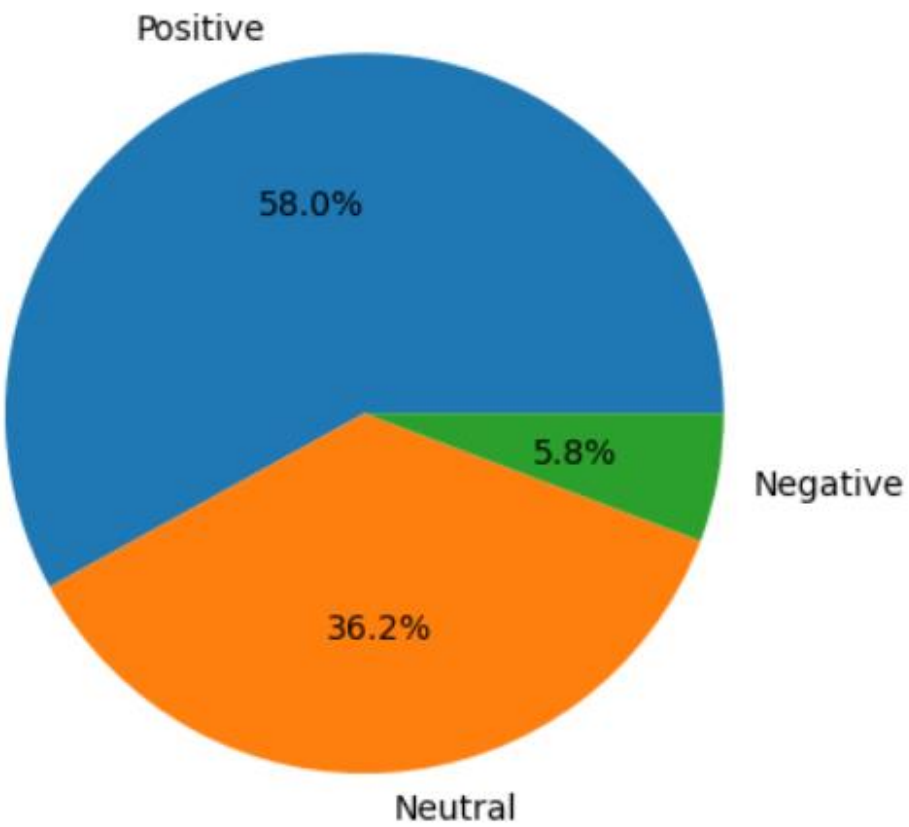
OUTPUT:

work
great
great
good
fast
card
sandisk
memory
perfect
works
storage
phone
price
excellent
sd card
samsung galaxy
micro sd
quality
working
great price
memory card
sandisk ultra
galaxy note
galaxy s4
galaxy s3
tablet
buy
uhs memory
reliable
device
bought
best
awesome
works fine five stars
five stars
nice
issue
failed
ultra gb
expected
needed
camera
speed
well
love
problem
far
good product
great product
good price
microsd card
month
adapter
class uhs
gb microsdhc
space
gopro
64gb
32gb
job
happy
fine
lot
one
little
amazing
super fast
huge
defective
small
new
capacity
large
value
photo
brand
hold
never
exactly
30mb
micro
ok
better
high speed
quick
class
amazon
fast reliable
microsdxc class
microsdhc class
surface pro
surface rt microsd
good value
great item
micro sdhc
gopro hero3
gopro hero4
galaxy tab
gb microsdxc
san disk
works perfectly
inexpensive
advertised
great works
smartphone
faster
cheap
worked
upgrade
review
samsung
performance
without
reliability
seemed
compatible
flawless
card 30mb
great deal
fantastic
solid
charm
fit
extra
complaints
disk
product
slow
described
dead
data
chip
bad
died
tiny
using
s4
music
addition
android phone
gopro hero3
microsdhc c10
gp
gig
micro sdhc
microsdxc class
fast reliable
amazon
class
good deal
two
device
reliable
uhs memory
galaxy s3
capacity
new
small
problem
defective
speed
well
love
price
super fast
huge
little
amazing
lot
tiny
died
bought
must
hero3
gopro hero
fantastic
card 30mb
great deal
say
will
s3
gs3
slow
described
surface pro
one
dead
data
chip
bad
galaxy note
year
used
money
purchase
large
value
photo
brand
hold
never
exactly
30mb
micro
ok
better
high speed
quick
class
amazon
fast reliable
microsdxc class
microsdhc c10
gp
gig

8. Perform sentiment analysis on the summary column using TextBlob and plot a pie chart showing the proportion of Positive, Neutral, and Negative sentiments.

```
from textblob import TextBlob

def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
        return "Positive"
    elif polarity == 0:
        return "Neutral"
    else:
        return "Negative"

df['sentiment'] = df['summary'].apply(get_sentiment)

sentiment_counts = df['sentiment'].value_counts()

plt.figure()
plt.pie(sentiment_counts, labels=sentiment_counts.index, autopct='%1.1f%%')
plt.show()
```

OUTPUT:



9. Perform topic modeling on the summary column using CountVectorizer and Latent Dirichlet Allocation (LDA) to extract the main topics and display the top words for each topic.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

vectorizer = CountVectorizer(max_df=0.9, min_df=5, stop_words='english')
doc_term_matrix = vectorizer.fit_transform(df['summary'])

lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(doc_term_matrix)

words = vectorizer.get_feature_names_out()

for i, topic in enumerate(lda.components_):
    print(f"Topic {i+1}:")
    print([words[j] for j in topic.argsort()[-10:]])
```

OUTPUT:

```
Topic 1:
['cards', 'far', 'value', 'quality', 'price', 'buy', 'product', 'gopro', 'sandisk', 'good']
Topic 2:
['bought', 'expected', 'working', 'stars', 'best', 'worked', 'use', 'months', 'fine', 'works']
Topic 3:
['class', 'micro', '10', 'gb', 'ultra', 'sandisk', 'sd', 'great', 'memory', 'card']
Topic 4:
['capacity', 'small', '64gb', 'problems', 'love', 'storage', 'just', 'reliable', 'perfect', 'fast']
Topic 5:
['s3', 'note', 'product', 'excellent', 'phone', 'samsung', 'price', 'galaxy', 'works', 'great']
```

Video Link –

https://youtu.be/UKQ17F0haEg

RESULT:

Based on the visualization and analysis of textual data, the programs were executed successfully and the outputs were obtained as expected.